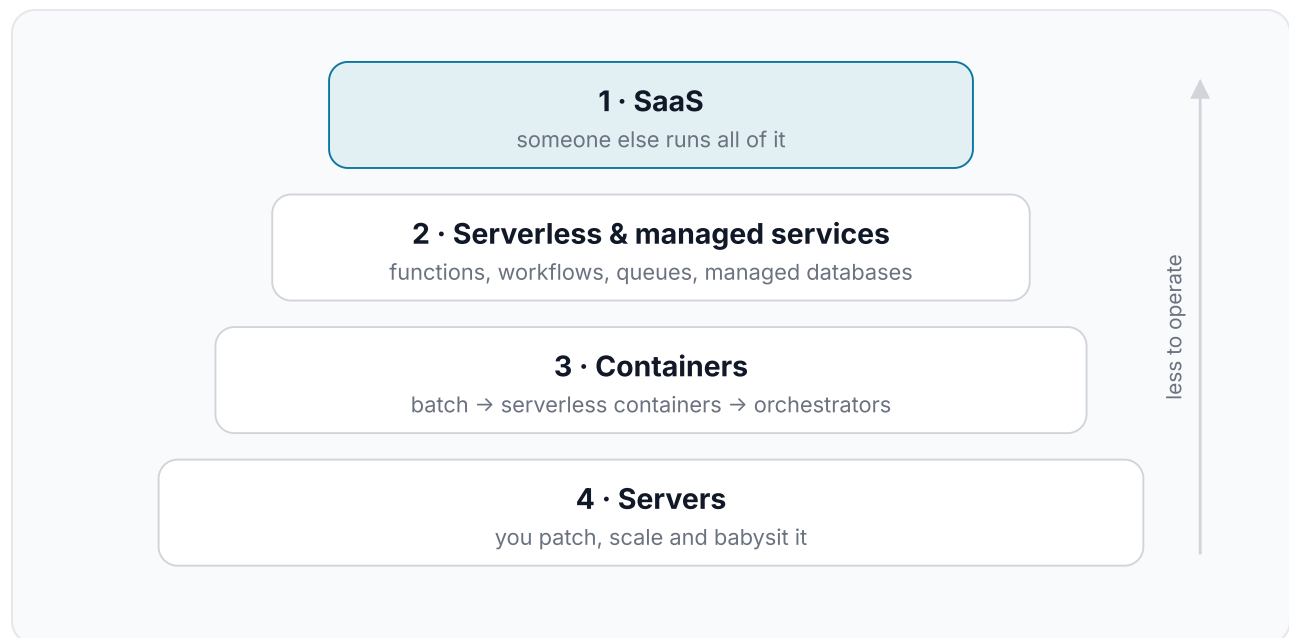


The Compute Ladder: Serverless-First, Then Work Backwards

A simple order of preference for where code runs — SaaS, serverless, containers, servers — why each rung up removes work, and how to design the ideal version first and only step down for a real constraint.

September 26, 2026 · 3 min read
<https://brianpfeil.com/architecture/compute-ladder/>

When a team asks “where should this run?”, I don’t start with their stack. I start at the top of a ladder and ask what stops us from staying there.



The rule

Design the ideal, most-managed version first. Then work backwards, one rung at a time, and only for a constraint you can name.

The mistake I see most often is the reverse: start from what the team already runs, and only move up if someone insists. That produces architectures shaped by habit rather than by the problem.

Why each rung up is worth it

Every step up the ladder hands a whole category of work to someone else:

Moving from...	...to	You stop owning
Servers	Containers	OS images, most patching, bin-packing by hand
Containers	Serverless	Cluster capacity, scaling policy, idle cost
Serverless	SaaS	The code itself

That's why I treat less-managed options as the ones that need justifying. The burden of proof sits with the servers, not the functions.

Within each rung, there's a ladder too

The same thinking applies inside a rung.

Serverless functions, most managed first:

1. The provider's managed runtime
2. The managed runtime plus layers for shared code
3. A container image as the function package
4. A custom runtime

Containers, most managed first:

1. A managed batch service, for work that runs and finishes
2. Serverless containers — no cluster to manage
3. A container orchestrator, when you genuinely need its control

What a "preferred" service looks like

When I'm choosing between services, the ones I reach for share four traits:

- **Pay-per-use** — no idle cost when nothing is happening.
- **Fully managed** — no patching, no capacity planning.
- **Highly available by default** — multi-AZ without extra work.
- **The provider carries most of the shared-responsibility model.**

The more of those a service has, the less your team has to build and run around it.

Services are Lego blocks

Serverless architecture is mostly **composition, not programming**. Think of a cloud provider's managed services the way you think of a language's standard library: you wouldn't write your own hash map, so don't write your own queue, retry loop or workflow engine.

Two habits follow from that:

- **Configuration over code.** If a service can do the job through configuration — a direct integration, a routing rule, a lifecycle policy — prefer that to a function. Code is a liability: it has to be tested, patched and owned.
- **Learn the blocks.** You can't compose what you don't know. Read the limits and quotas page of every service *before* you design around it; that's where most surprises live.

Good reasons to step down a rung

Stepping down isn't failure. These are constraints I'd accept:

- **Execution time or resources** beyond what functions allow.
- **A vendor product** that only ships as a container or a server image.
- **Real local state** or specialised hardware.
- **A proven cost crossover** at sustained high load — with the numbers, and including the operations cost you're taking back on.

And ones I wouldn't: familiarity, "we might need it later", or a design that only needs a server because it polls on a timer (make it [event-driven](#) instead).

If you do step down, record why in a [decision record](#) and give the next step up a date. Today's constraint is often next year's new managed feature.